Tecnológico de Monterrey

E2. Integrative Project

Object-Oriented Programming (Gpo 304)

Mauricio Rey Hernández Curiel - A01648241

Professor: Carlos Ramón Garibay Guémez

Tecnológico de Monterrey campus Guadalajara

June 10th, 2026

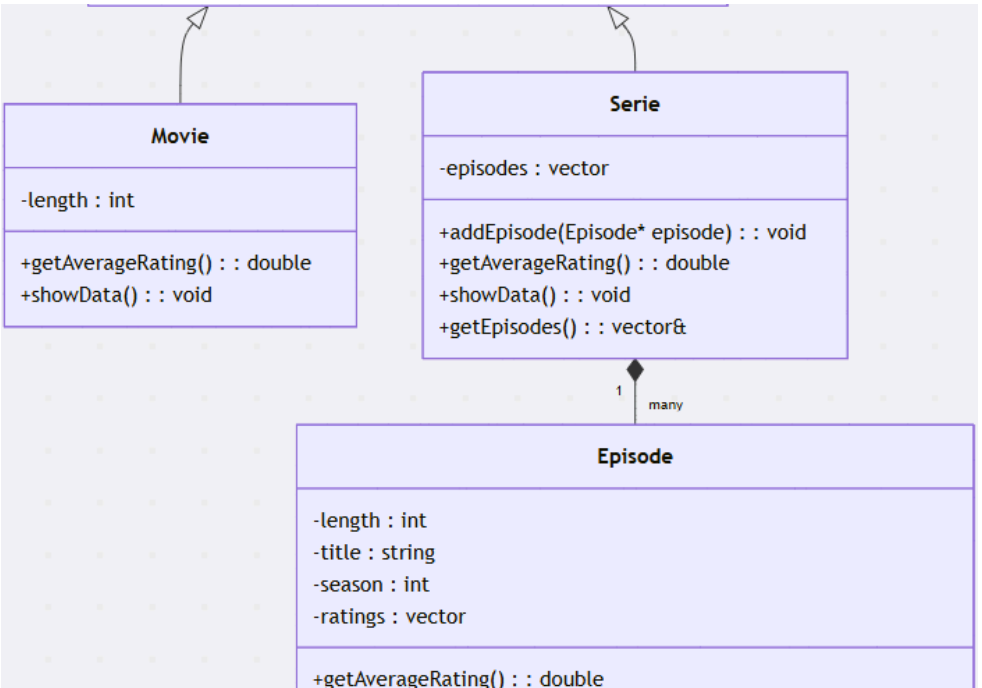
Content index:

Introduction: Problem Statement..... 3
UML class diagram - Link to mermaidviewer for better visualization.....4
UML diagram justification.....5
Execution Code Video:.....6
Cases that would prevent the project from functioning properly.....6
Personal conclusion.....6

Introduction: Problem Statement

The rapid rise of on-demand streaming platforms like Netflix, Disney+, Max, and others has created a need for software systems that can organize, present, and rate large catalogs of multimedia content. This project addresses a simplified version of such a system, implementing an object-oriented solution in C++ that manages two types of video content: movies and series using classes, while ensuring that shared behaviour is captured through inheritance and that it is handled through polymorphism.

UML class diagram - [Link to mermaidviewer for better visualization]



UML diagram justification

Video as an abstract class because no object is “just a video”, it is either a movie or a series. The attributes are declared as private so that subclasses can inherit them directly without needing getters, aswell getAverageRating() and showData() are pure virtual because their implementation is fundamentally different for each type of video. And the operator<< is defined as a friend function so that any Video pointer can be printed through cout << *pointer instead of calling a method.

Movie inherits Video and only adds one attribute, length, and the virtual methods are overridden here to provide movie-specific formatting and rating calculation.

Serie inherits Video, its key distinction is that it owns a vector of Episode pointers. getAverageRating() is overridden to aggregate the ratings of all its episodes, skipping unrated ones to avoid dragging the average down. getEpisodes() returns a reference to the internal vector so Catalog can iterate episodes without exposing the vector by value.

Episode has a composition relationship with Serie because episodes have no existence outside their Serie, if a Serie is destroyed, its episodes are destroyed with it. Episode lacks an id and a genre because those belong to the Serie they are part of. Episode is its own concrete class with its own ratings vector, operator<<, and getAverageRating() because I think it was the best way to keep the Video class clean and have less trouble handling.

Catalog has an aggregation relationship with Video because the videos are the content being managed, not parts that exist only because of Catalog. Catalog is a container that organizes videos, and the private helpers getMovies() and getSeries() are included in the diagram because they are meaningful collaborators that use dynamic_cast to extract specific

videos, withinRange() and toLower() appear as private static helpers because they are pure utility functions with no dependency on instance state.

Overall the diagram is not complex and uses polymorphism and inheritance correctly.

Execution Code Video:

Video link for the execution example

Cases that would prevent the project from functioning properly

Table 1. *Scenarios that prevent the project from functioning properly.*

Scenario	Impact
Dataset file not found or wrong path or nonexistent	Code prints 'Error opening file'. Can't do any rating or search
File without same syntax or termination (I used .txt)	They could either produce garbage values, or an error and there's no catch block to handle this.
Rating a video with a rate that isn't in the set of rates	Just a retry error message, but the rating will not pass.
Searching before loading data	Code will always report not found.
Duplicate titles	Not to important, but will print both when searching if they share the same rating

Personal conclusion

Overall I enjoyed making this project although it took a little more time than I expected. I learned a lot about OOP, new features such as what is a friend function or a dynamic_cast and how they work, as well I learned more about how cin works and that you

can clear it and discard characters from the input buffer. Designing the whole class structure and watching how it slowly starts taking shape and at the end compiles is a satisfactory experience.

Although I can see that there may be things to make better, like the set validation for ratings because there's probably a better way to do it but I don't quite know how to handle floats, I also think a try and catch block can be applied in some cases for error handling especially for loading the data set.

Episode presented an interesting dilemma because should it inherit from Video? And my decision was made to keep Episode as a standalone class because episodes do not have ids or genres, they belong entirely to a Serie and are only important in that context. Forcing them into the Video hierarchy would have polluted the base class with unused or defaulted attributes. The trade-off is that Episode cannot be stored in the vector and requires its own operator<< overload, which is a small but acceptable cost.

This project successfully satisfies all requirements, the class hierarchy is minimal, the code works with polymorphism and the menu application validates user input at every entry point. The code is readable, commented, and I think it demonstrates solid OOP concepts including inheritance, polymorphism, operator overloading, dynamic_cast and friend functions. It represents the best solution achievable within the objective and constraints of the assignment.